上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

命题逻辑可满足性求解算法

符鸿飞

计算机科学与工程系

上海交通大学

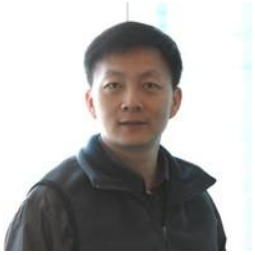
Slides based on 熊英飞@北京大学

历史

- 命题逻辑求解算法历史上一直有两个特点：
  - 速度慢
- 进入2000年以来
  - SAT的求解速度得到了突飞猛进的进步
    （理论上还无法完全解释SAT求解的高速表现）

Sharad Malik
领导开发Chaff求解器
Lintao Zhang导师

Joao Marques-Silva
关键算法CDCL提出人



Lintao Zhang
CDCL的通用化和效率提升

SAT问题

- 最基本的NP完全问题（1971）

- 文字 literal：布尔变量 x 或 x的否定（即¬x）
- 子句 clause：有限个文字的析取，
  如 x∨(¬y)
- 合取范式：有限个子句的合取，
  如 (x∨(¬y))∧((¬x)∨z)
- 布尔赋值：从布尔变量到真值的一个赋值函数

◆SAT问题：给定一个合取范式，判断其是否可满足，意即是否存在一个布尔赋值，使得该合取范式中每一个子句均为真。

SAT问题举例

- 布尔变量用数字1~4表示
- 子句用布尔变量的子集表示
- 子集表示中，正负表示以变量本身还是以其否定形式出现
- {1，−4}
- {−2，3}
- {2，4}
- {−2，−3，4}
- {−1，−4}
- 该SAT问题是否可满足？
  - 不可满足（为什么？）
- 可满足条件：存在一组解，使得每个子句中至少有一个文字为真

SAT问题基本求解算法——穷举

```
SAT(assign) {
    if (assign是完整的) {
        if (整个合取范式为真) return true
        else return false
    else
        选择一个还未被赋值的布尔变量x;
        return
              SAT(assign ∪ [x→true])
            || SAT(assign ∪ [x→false])
    }
}
```

- NP完全问题
- 理论上公认不存在高效的算法

算法优化思想：冲突检测

- assign={1， 4}
- 红色为假，绿色为真
- {1， −4}
- {−2， 3}
- {2， 4}
- {−2， −3， 4}
- {−1， −4} //出现冲突
- 不需要完整赋值就能知道结果

SAT问题求解算法——冲突检测

```
SAT(assign) {
    if (assign有冲突) return false;  //冲突检测
    if (assign是完整的) return true;
    选择一个还未被赋值的布尔变量x;
     return
          SAT(assign ∪ [x→true])
       || SAT(assign ∪ [x→false])
}
```

算法优化思想：赋值推导

- assign={2}
- 红色为假，绿色为真
- {1, -4}
- {-2, 3}　　//代入
- {2, 4}　　//代入，此子句可删除
- {-2, -3, 4}　//代入
- {-1, -4}

算法优化思想：赋值推导

- assign={2}
- 红色为假，绿色为真
- {1，−4}
- {−2，3}　　//赋值推导
- {2，4}
- {−2，−3，4}　　//代入
- {−1，−4}

算法优化思想：赋值推导

- assign={2}
- 红色为假，绿色为真
- {1, -4}　　//代入
- {-2, 3}
- {2, 4}　　//代入
- {-2, -3, 4}　//赋值推导
- {-1, -4}　　//代入

算法优化思想：赋值推导

- assign={2}
- 红色为假，绿色为真
- {1, -4}　　//赋值推导
- {-2, 3}
- {2, 4}
- {-2, -3, 4}
- {-1, -4}　　//代入并导出矛盾

- 优化：无需继续遍历赋值就能得出结果

赋值推导标准方式

- **Unit Propagation**
  - 其他文字都为假，剩下的一个文字必定为真
  - {−1，2，−3} => 3
- **Unate Propagation**
  - 当一个子句存在为真的文字时，可以从子句集合中删除
    - {2，4}
- **Pure literal elimination**
  - 当一个变量只有全部为变量形式出现或者全部为否定形式出现的时候，可以把包含该变量的子句删除
    - {−4}
    - {−4，6}
    - {−4，−6}

基于赋值推导和冲突检测的DPLL

```
DPLL(assign) {
    assign'=赋值推导(assign);          //赋值推导
    if (assign'有冲突) return false;  //冲突检测
    if (assign'是完整的) return true;
    选择一个还未被赋值的布尔变量x;
     return
          DPLL(assign'∪[x→true])
       || DPLL(assign'∪[x→false])
}
```

- 该算法被称为DPLL，由Davis, Putnam, Logermann, Loveland在1962年代提出

高级推导方法

- Probing
  - 如果令x=false或者x=true都能推导出y=false，则推导出y=false
- Equivalence classes
  - 预先检查出等价的子句集合，然后删除其中一个
    - {1，2，−3}
    - {2，1，−3}

赋值推导中的变量选择

- 先选择哪个变量赋值可能对求解造成很大影响
  - {1, -2}
  - {1, 2}
  - {-1, -2}
  - {-1, 2}
  - {3, 4, 5, 6, 7, 8}
- 优先选择1或者2可以快速发现不可满足
- 优先选择3-8需要反复回溯多次

启发式变量选择方法

- 基于子句集的
  - 优先选择最短子句里的变量
  - 优先选择最常出现的变量
- 基于历史的
  - 优先选择之前导致过冲突的变量

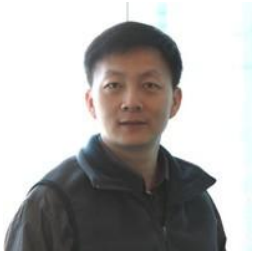
冲突导向的子句学习

Conflict Driven Clause Learning (CDCL)

- 2000年初期大幅提升SAT效率的重要因素之一
- 基本思想：在搜索过程中学习问题的性质，加入约束集合中

Sharad Malik
领导开发Chaff求解器
Lintao Zhang导师

Joao Marques-Silva
关键算法CDCL提出人

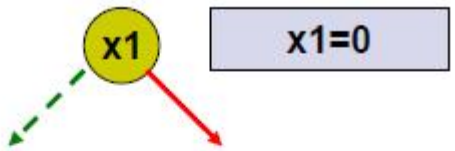


Lintao Zhang
CDCL的通用化和效率提升

一个DPLL实例

Step 1

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

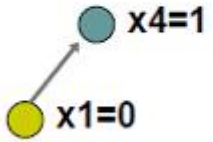


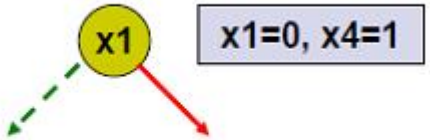
x1=0

一个DPLL实例

Step 2

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

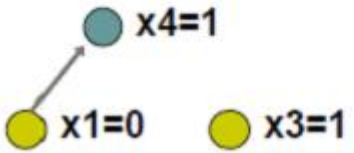


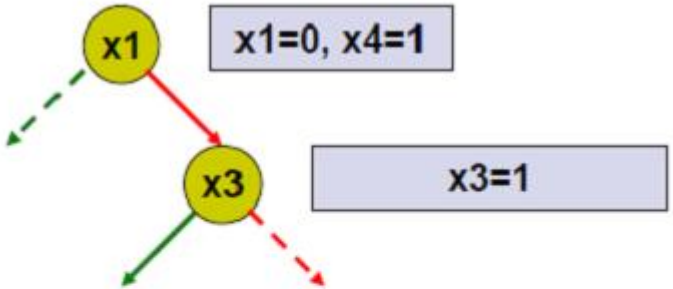


一个DPLL实例

Step 3

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

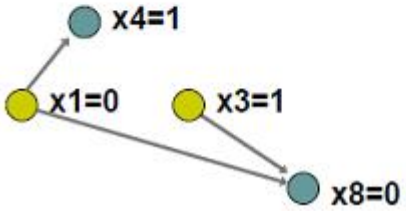


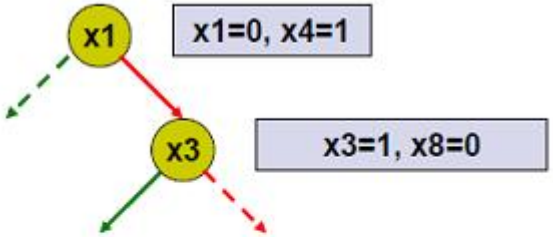


一个DPLL实例

Step 4

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

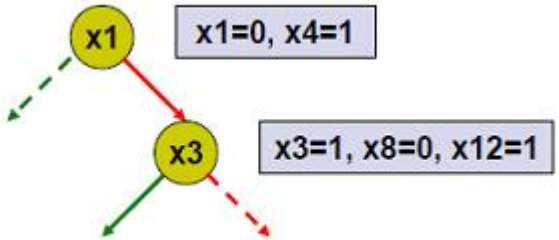


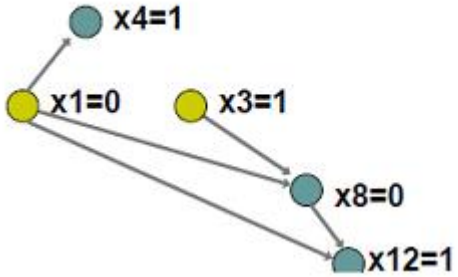


一个DPLL实例

Step 5

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

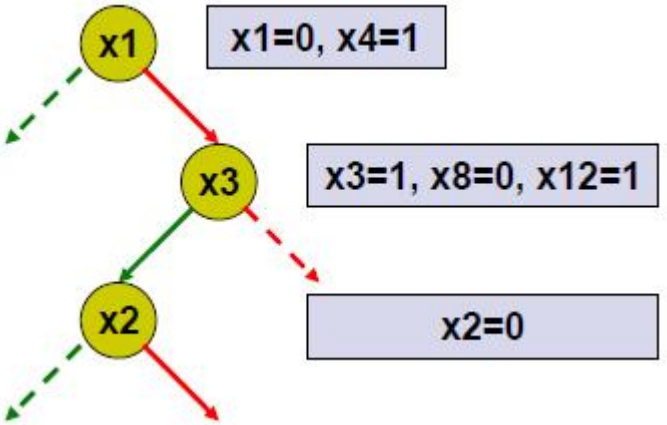


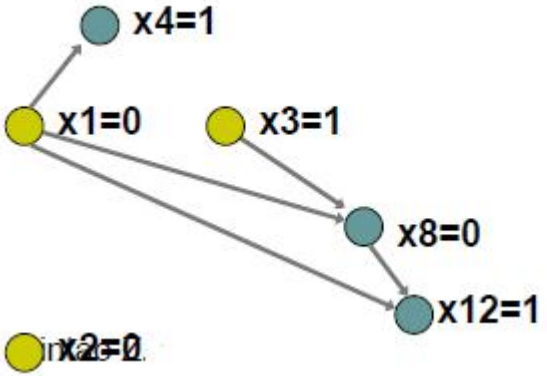


一个DPLL实例

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

Step 6

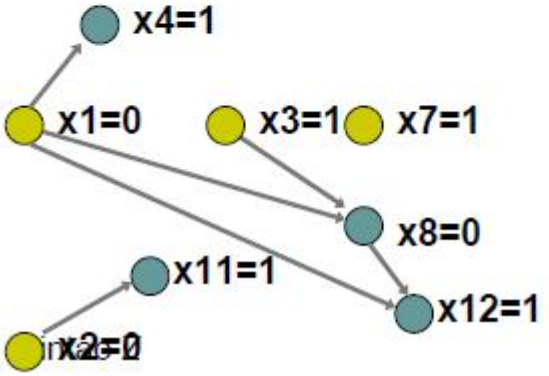


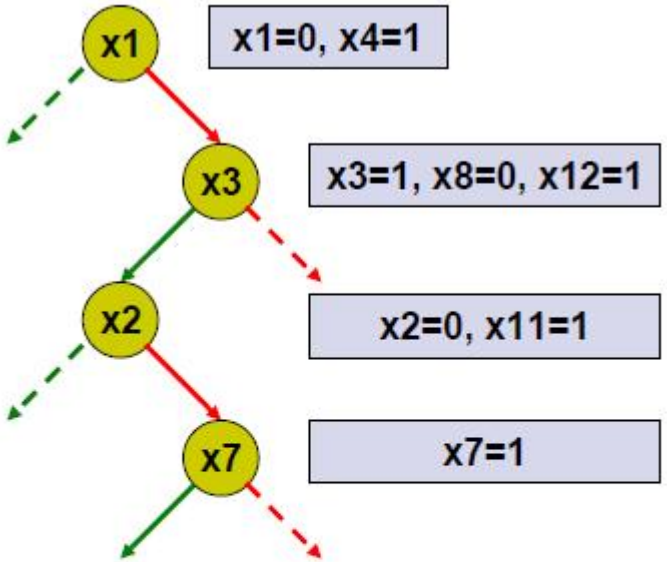


一个DPLL实例

Step 9

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

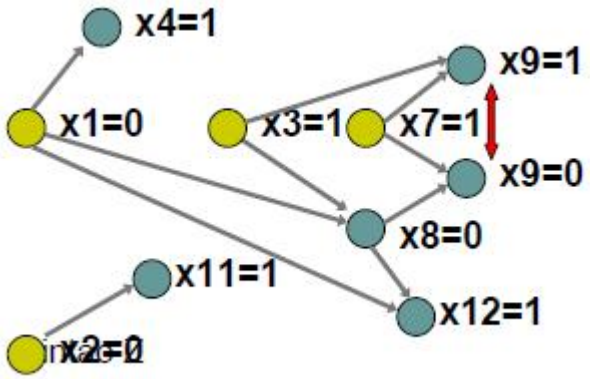


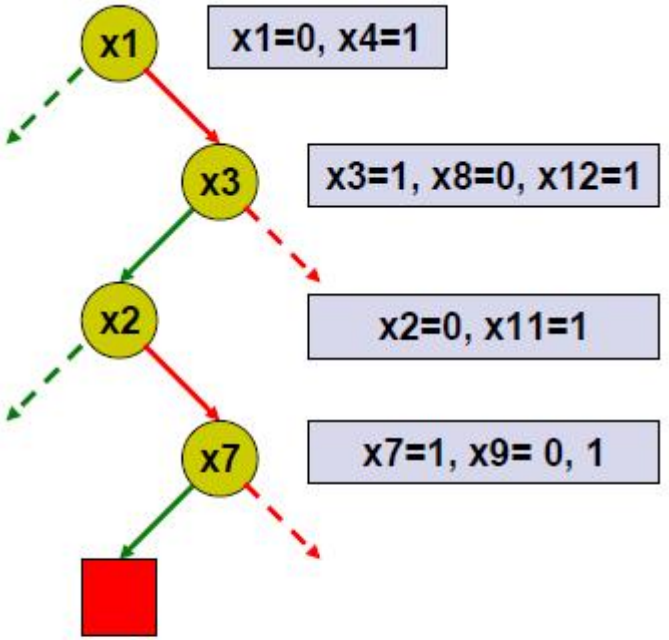


一个DPLL实例

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

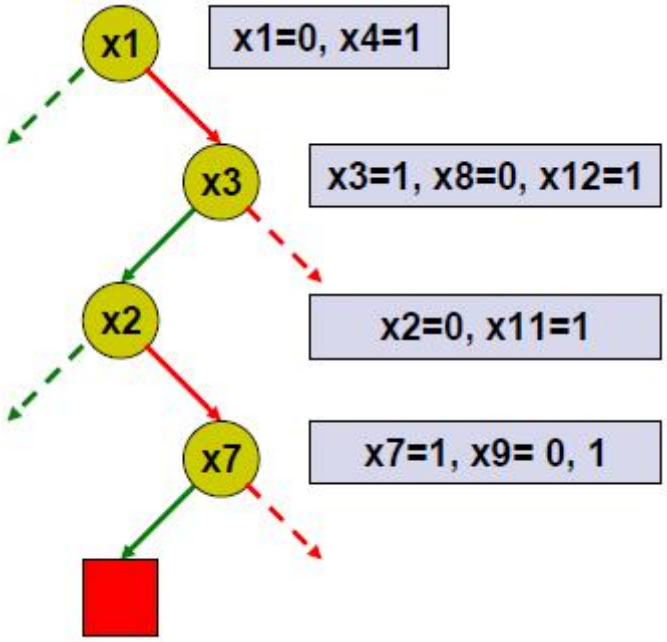
Step 10





一个DPLL实例

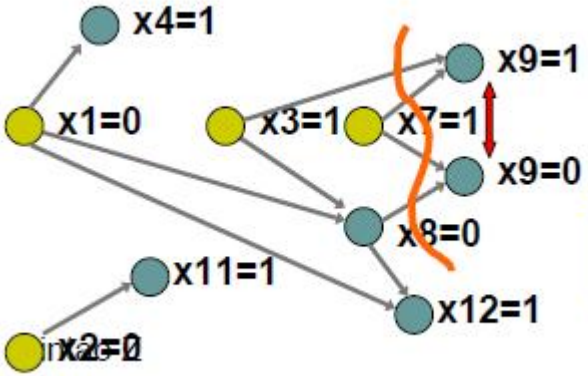
- 如果后续搜索把x7再次设置成了1，会重复出现该冲突

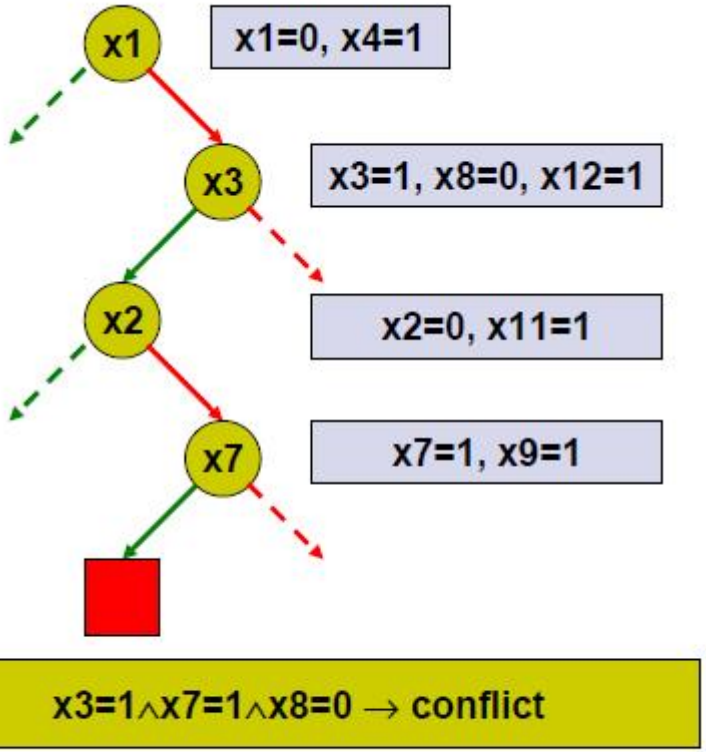


CDCL子句学习

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

Step 11

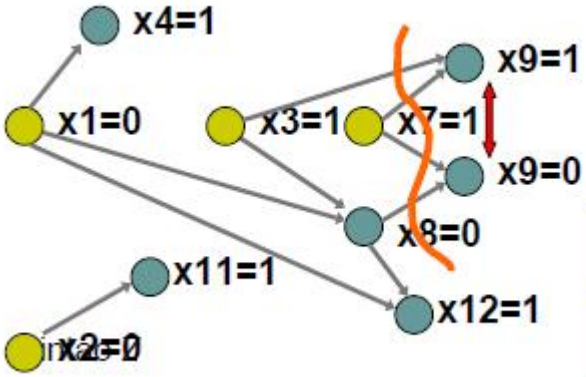


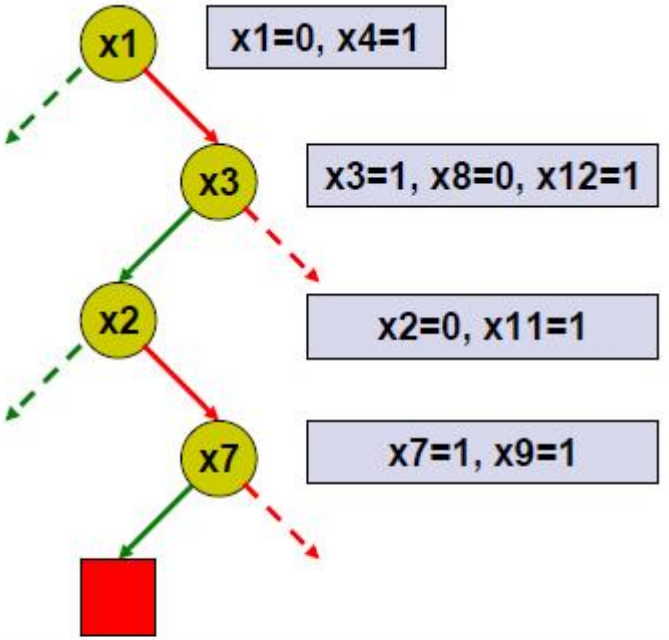


CDCL子句学习

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'

Step 13





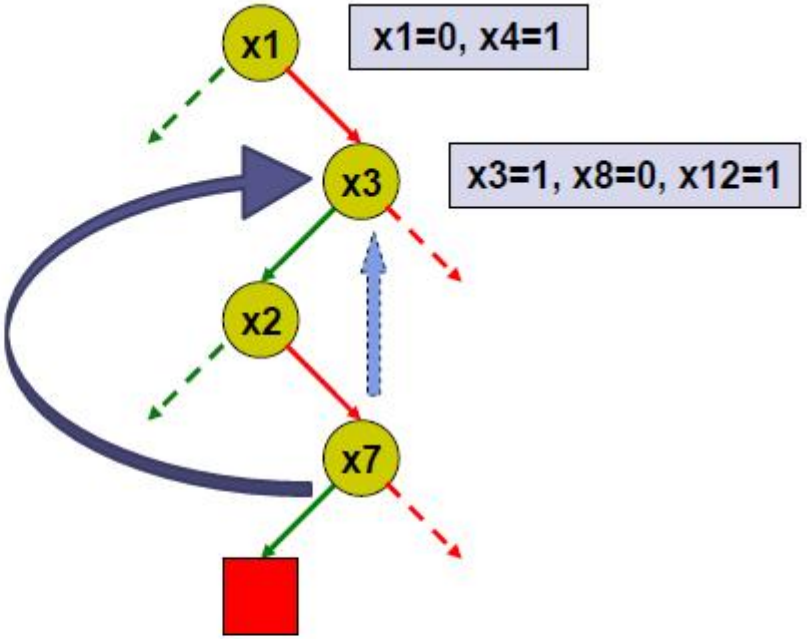
$x3=1 \wedge x7=1 \wedge x8=0 \rightarrow$ conflict

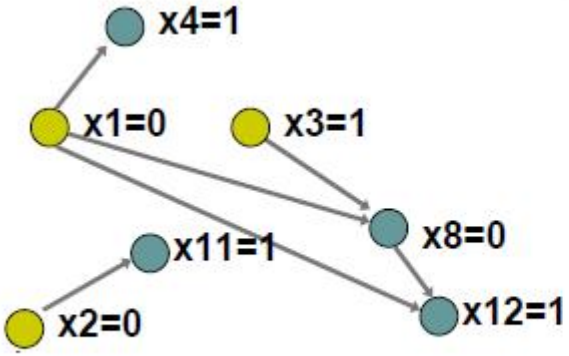
Add conflict clause: x3'+x7'+x8

CDCL子句学习

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'
x3' + x8 + x7'

Step 14



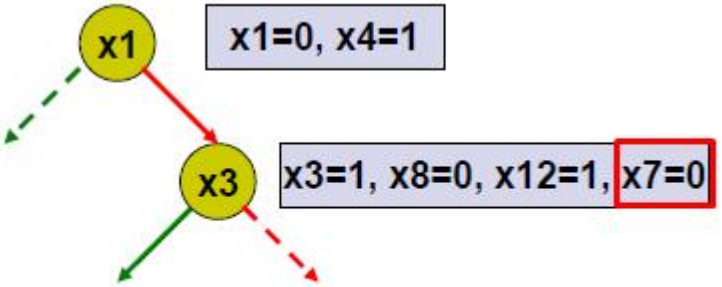


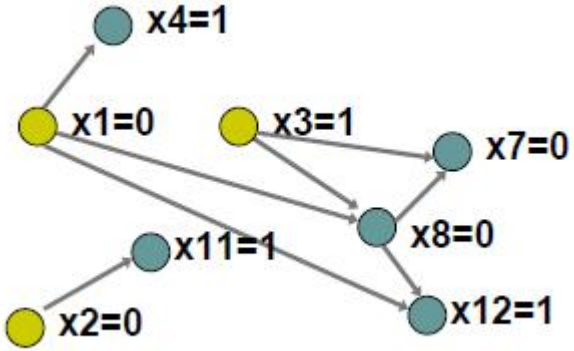
Backtrack to the decision level of x3=1:
x7 = 0

CDCL子句学习

x1 + x4
x1 + x3' + x8'
x1 + x8 + x12
x2 + x11
x7' + x3' + x9
x7' + x8 + x9'
x7 + x8 + x10'
x7 + x10 + x12'
x3' + x8 + x7'

Step 15





CDCL子句学习

- 从新添加约束出发的推导实际保证了之前探过的冲突赋值不会出现
- 所以不再需要记录之前遍历过的赋值，每次任意选择剩下的变量和赋值即可

CDCL子句学习算法

```
CDCL(assign) {
    assign=空赋值;
    while (true) {
        if (当前变元的所有赋值均被尝试) {
                改变上一次赋值;  //回溯
                 if (回到空赋值) reture false;};
        赋值推导(assign);
        if (推导结果有冲突) {
            if (assign为空) return false;
            添加新约束;    //子句学习
            撤销当前赋值;        //回溯
        } else {
            if (推导结果是完整的) return true;
            选择一个未尝试的赋值添加到assign;
        }
}
```

总结

- SAT问题的求解算法
  - 穷举法
  - 冲突检测
  - 赋值推导
  - DPLL算法
  - CDCL方法

问题时间